CEN3024C Module 6 Project

For this project we will be implementing a CORS policy at the global level and a "one-off" method-level policy to the Task Manager project. We will also be adding support for a password for the users.

1. Add a global policy using a configuration class (add this in a new "config" package):

```
@Configuration
public class CorsConfig implements WebMvcConfigurer {
    @Override
    public void addCorsMappings(CorsRegistry registry) {
        registry.addMapping("/**")
            .allowedOrigins("http://example.com")
            .allowedMethods("GET", "POST", "PUT", "DELETE")
            .allowedHeaders("*")
            .allowCredentials(true);
    }
}
```

2. Choose an endpoint in your controllers and add a method-based policy:

```
@CrossOrigin(origins = {"http://example.com", "http://localhost"})
// Get all users
@GetMapping
...
```

I added mine to the top level endpoint in the User class. Technically it doesn't matter if the @CrossOrigin annotation preceeds or follows the @GetMapping annotation, but convention dictates that it should go before @GetMapping.

3. Test and verify the correct operation of your policies with curl by specifying the Origin field.

Any request using the example.com origin should return a 200 and display the corresponding data,

e.g.,

```
curl -i -H "Origin: http://example.com" -X GET http://localhost:8080/users
...
HTTP/1.1 200
```

But a request from example2 should not:

```
curl -i -H "Origin: http://example2.com" -X GET http://localhost:8080/users
...
HTTP/1.1 403
```

```
Invalid CORS request
```

3. Add a salt and hash field to the user entity and a password field to the customer DTO. Modify the UserService class to implement the salt and hash operations using PBKDF2WithHmacSHA1 (a popular algorithm recommended by NIST), or you can try a different algorithm if you like. Verify that your POST saves the salt and hash values to the database.

4. Submit your solution to the GitHub classroom assignment repo.